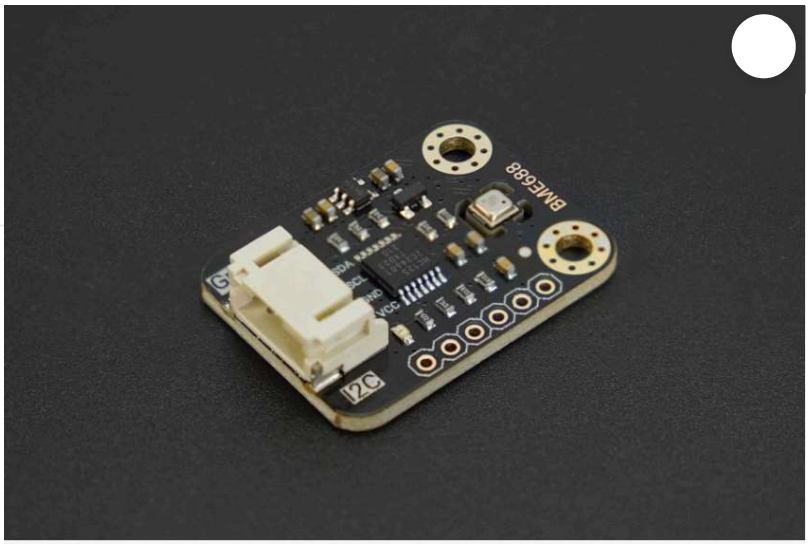
SKU:SEN0617 (https://www.dfrobot.com/product-2871.html)

(https://www.dfrobot.com/product-2871.html)



Brief

DFRobot Gravity BME688 Environmental Sensor is a highly integrated MEMS environmental sensor capable of comprehensively measuring environmental parameters such as temperature, humidity, barometric pressure and VOC index (volatile organic compounds). The software and hardware dimensions are fully compatible with the formal BME680, making it a drop-in replacement for the BME680 in existing systems without the need for any adjustments, while the BME688 improves the accuracy of the temperature measurements and skeletonizes the periphery of the IC on the circuit board to minimize the effect of heat from external components on the measurements.

Another major improvement of the BME688 is the introduction of a “heating layer” on the bottom of the chip. While conventional VOC sensors are susceptible to ambient temperature interference, leading to measurement errors, the BME688's heating layer allows the user to precisely control the sensor's temperature, thus minimizing the impact of ambient temperature on VOC measurements. With this feature, users can also adjust the temperature of the heating layer to collect data on the response of specific composite gases at different temperatures for in-depth analysis in conjunction with AI technology.

Note: If you hope to use the AI function of the BME688 for gas analysis, you will need to use BME AI-Studio software (https://www.bosch-presse.de/pressportal/de/en/bme-ai-studio-software-225349.html) and BSEC2-Arduino library (https://github.com/BoschSensortec/Bosch-BSEC2-Library) released by Bosch for secondary development

Specification

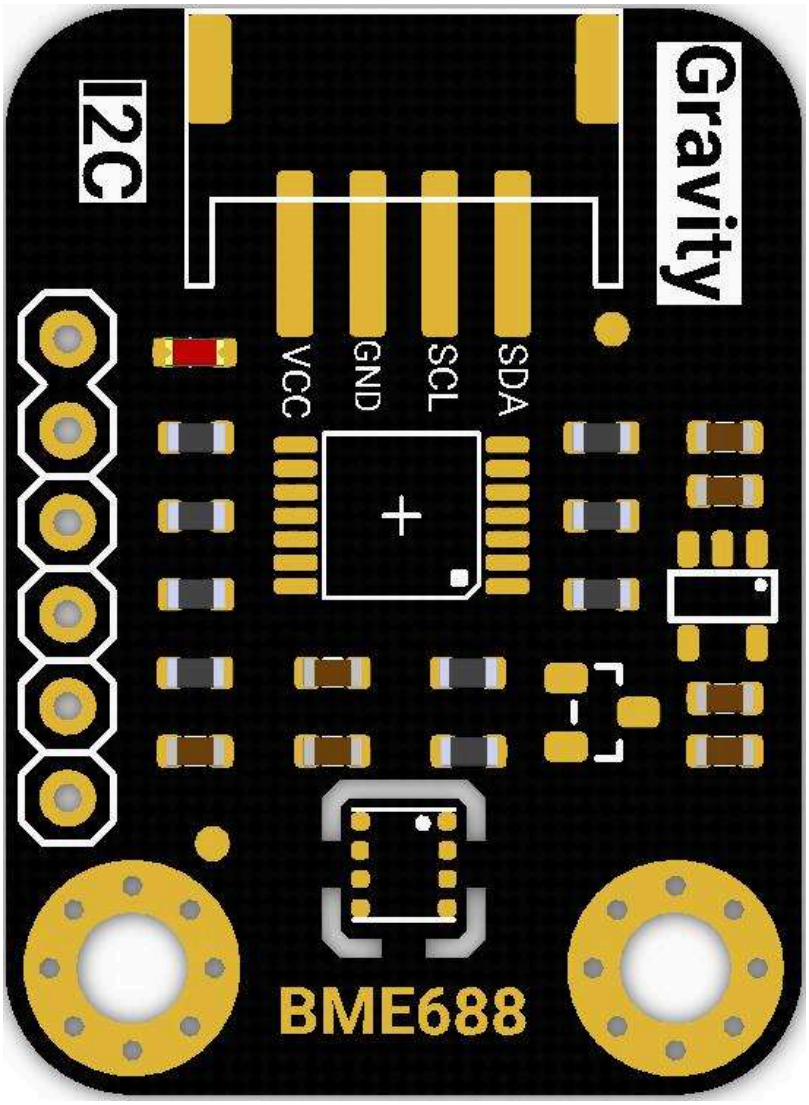
- Working voltage：3.3V-5.0V
- Working current：5mA（Operating current of 25mA with VOC measurement turned on）
- Communications interface：Gravity I2C
- Reserved interface：SPI

- Reserved interface：SPI
- Temperature measurement range：-40℃~+85℃
- Temperature measurement accuracy：±0.5℃(0~65℃)
- Humidity measurement range：0-100%r.H.
- Humidity measurement accuracy：±3%r.H.(20-80% r.H.,25℃)
- Barometric Pressure Measurement Range：300-1100hPa
- Barometric Pressure Measurement accuracy：±0.6hPa(300-1100hPa,0~65℃)
- IAQ（Indoor Air Quality）range：0-500（The higher the value, the worse the air quality）
- Module Size：30 × 22(mm) / 1.18 x0.87(inches)

IAQ(Indoor Air Quality) Comparison Table

IAQ Index	Air Quality
0 – 50	good[10]
51 – 100	average
101 – 150	little bad
151 – 200	bad
201 – 300	worse[2]
301 – 500	very bad

Pinout



Pinout Descripton

Pinout	Name	Description
+	VCC	3.3~5V
-	GND	GND
C	SCL	I2C-SCL
D	SDA	I2C-SDA

Tutorial

This tutorial will demonstrate how to use this sensor module. Currently, only FireBeetle ESP8266 (https://www.dfrobot.com/product-1634.html) can read the IAQ value, other host controllers are not supported yet.

Prepare

Hardware

- DFRduino UNO (https://www.dfrobot.com/product-838.html) x1
- Gravity: BME688 environmental sensor (https://www.dfrobot.com/product-2871.html) x1
- Gravity IO Expansion shield (https://www.dfrobot.com/product-1009.html) x1
- Jumper Wire(F/F) (https://www.dfrobot.com/product-356.html) for SPI connection x1

Software

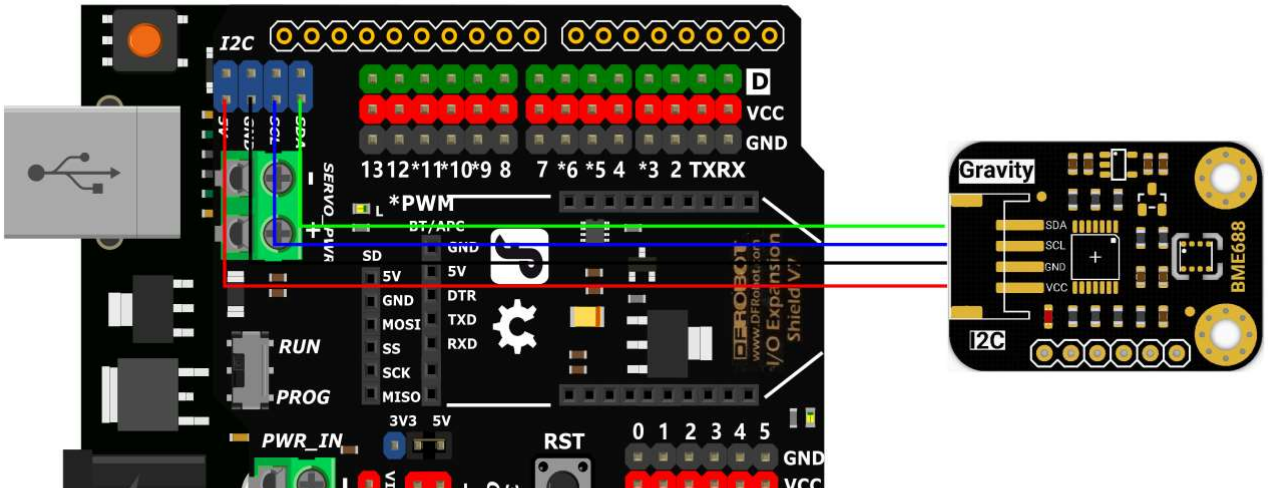
- Arduino IDE (1.0.x or 1.8.x)， Arduino IDE (https://www.arduino.cc/en/Main/Software)
- Download Arduino IDE，DFRobot_BME68x (https://github.com/DFRobot/DFRobot_BME68x) How to install

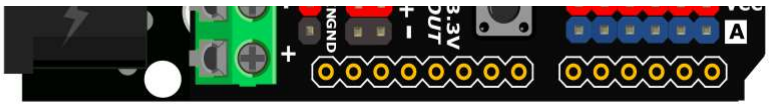
Wiring Diagram

- **This product supports I2C interface and SPI interface. When using this product, please select the corresponding wiring method according to the communication interface selected. Refer to the following diagram for the connection method.Please refer to the following diagram for the connection method.**
- **I2C interface is recommended for plug-and-play, easy to use. It is recommended to use the I2C interface, which is easy to use.**
- If SPI interface is used, the module will be powered by 3.3V.

I2C diagram

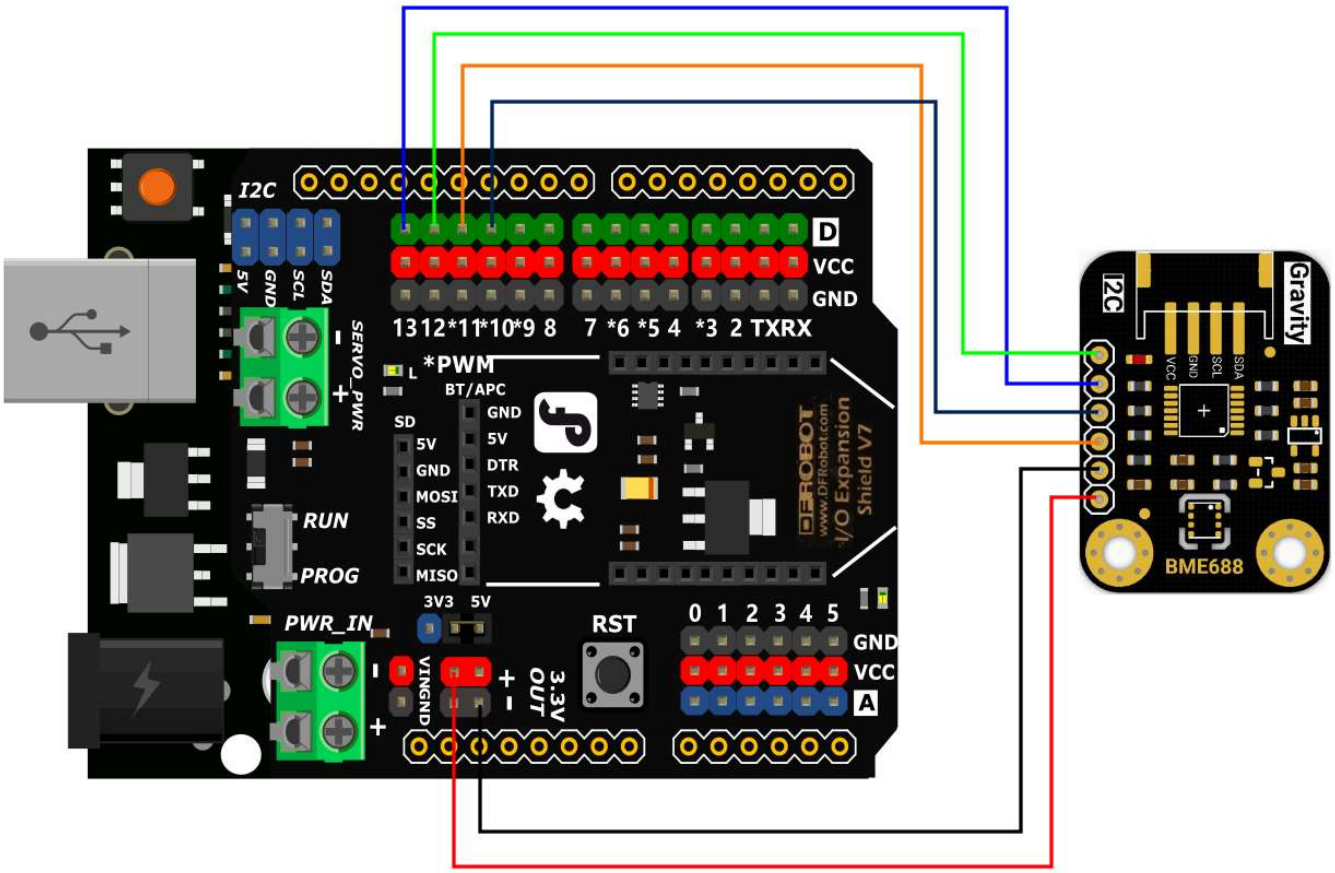
Be sure to pay attention to the wire sequence, VCC to 5V, GND grounded





SPI diagram

Be sure to pay attention to the wire sequence, VCC to 3.3V, GND grounded



Sample code

Please install the Arduino library files first. DFRobot_BME68x (https://github.com/DFRobot/DFRobot_BME68x) This sample code uses I2C interface. For SPI interface sample code, please go to the library file to find DFRobot_BME68x_SPI.ino file, due to the same function, we do not show the code here. **Calibration is required for accurate altitude measurement. Before uploading the code, please fill in the altitude value of your region into the statement in the sample code: seaLevel = bme.readSeaLevel(altitude value of your region);**

Reading data (without IAQ index)

Program Function: Reads sensor data and performs serial port printing. (without IAQ) Set the heating layer to 320℃ for 100ms through setGasHeater() function. and wait for 30ms (the heating layer needs 20~30ms to be heated to the target temperature). And read the gas resistance value by readGasResistance() function.

```
/*!
 * @file DFRobot_BME68x_I2C.ino
 * @brief connect bme68x I2C interface with your board (please reference board compatibili
 * @n Temprature, Humidity, pressure, altitude, calibrate altitude and gas resistance data
 *
 * @copyright   Copyright (c) 2010 DFRobot Co.Ltd (http://www.dfrobot.com)
 * @license     The MIT License (MIT)
 * @author [Frank](jiehan.guo@dfrobot.com)
 * @maintainer [GDuang](yonglei.ren@dfrobot.com)
 * @version  V2.0
 * @date  2024-04-25
 * @url https://github.com/DFRobot/DFRobot_BME68x
 */

#include "DFRobot_BME68x_I2C.h"
#include "Wire.h"

/*use an accurate altitude to calibrate sea level air pressure*/
#define CALIBRATE_PRESSURE

DFRobot_BME68x_I2C bme(0x77);  //0x77 I2C address

float seaLevel;
void setup()
{
  uint8_t rslt = 1;
  Serial.begin(9600);
  while(!Serial);
  delay(5000);
  Serial.println();
  while(rslt != 0) {
    rslt = bme.begin();
    if(rslt != 0) {
      Serial.println("bme begin failure");
      delay(2000);
    }
  }
  Serial.println("bme begin successful");
  #ifdef CALIBRATE_PRESSURE
  bme.startConvert();
  delay(1000);
  bme.update();
  /*You can use an accurate altitude to calibrate sea level air pressure.
   *And then use this calibrated sea level pressure as a reference to obtain the calibrate
   *In this case,525.0m is chendu accurate altitude.
   */
  seaLevel = bme.readSeaLevel(525.0);
```

```
  Serial.print("seaLevel :");
  Serial.println(seaLevel);
  #endif

  // At initialization, the default heating layer target temperature is 320 and the durati
  bool res = bme.setGasHeater(360, 100);

  Serial.print("Set the target temperature of the heating layer and the heating time: ");
  if(res == true){
    Serial.println("set successful!");
  }else{
    Serial.println("set failure!");
  }
}

void loop()
{
  bme.setGasHeater(320, 100);
  delay(30);
  bme.startConvert();
  delay(1000);
  bme.update();
  Serial.println();
  Serial.print("temperature(C) :");
  Serial.println(bme.readTemperature() / 100, 2);
  Serial.print("pressure(Pa) :");
  Serial.println(bme.readPressure());
  Serial.print("humidity(%rh) :");
  Serial.println(bme.readHumidity() / 1000, 2);
  Serial.print("gas resistance(ohm) :");
  Serial.println(bme.readGasResistance());
  Serial.print("altitude(m) :");
  Serial.println(bme.readAltitude());
  #ifdef CALIBRATE_PRESSURE
  Serial.print("calibrated altitude(m) :");
  Serial.println(bme.readCalibratedAltitude(seaLevel));
  #endif
}
```

Reading data (with IAQ values)

Program Function: Reads sensor data and performs serial printing. (with IAQ) Currently, only FireBeetle ESP8266 (https://www.dfrobot.com/product-1634.html) can read IAQ values, other dev board are not supported at this time. For FireBeetle ESP8266 WiFi IoT development board, please use Arduino IDE version 1.8, and you need to upgrade the SDK to 2.3.1 and above. Upgrade method reference FireBeetle ESP8266 wiki (https://wiki.dfrobot.com/FireBeetle_ESP8266_IOT_Microcontroller_SKU__DFR0489#target_3)

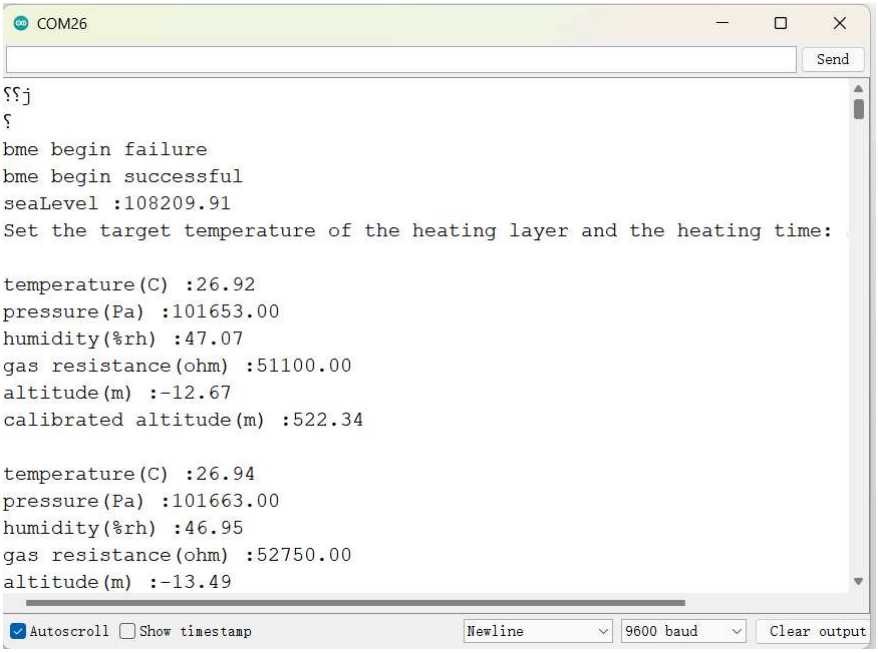
```
/*!
 * @file IAQ_I2C.ino
 * @brief connect bme68x I2C interface with your board (please reference board compatibili
 * @n Temprature, Humidity, pressure, altitude, calibrated altitude, gas resistance and IA
 * @note This demo only support ESP8266 MCU
 *
 * @copyright   Copyright (c) 2010 DFRobot Co.Ltd (http://www.dfrobot.com)
 * @license     The MIT License (MIT)
 * @author [Frank](jiehan.guo@dfrobot.com)
 * @maintainer [GDuang](yonglei.ren@dfrobot.com)
 * @version  V2.0
 * @date  2024-04-25
 * @url https://github.com/DFRobot/DFRobot_BME68x
 */

#include "DFRobot_BME68x_I2C.h"
#include "Wire.h"

/*use an accurate altitude to calibrate sea level air pressure*/
#define CALIBRATE_PRESSURE

DFRobot_BME68x_I2C bme(0x77);  //0x77 I2C address


float seaLevel;
void setup()
{
  uint8_t       rslt = 1;
  Serial.begin(115200);
  while(!Serial);
  delay(5000);
  Serial.println();
  while(rslt != 0) {
    rslt = bme.begin();
    if(rslt != 0) {
      Serial.println("bme begin failure");
      delay(2000);
    }
  }
  Serial.println("bme begin successful");
  bme.supportIAQ();

  // At initialization, the default heating layer target temperature is 320 and the durati
  bool res = bme.setGasHeater(360, 100);

  Serial.print("Set the target temperature of the heating layer and the heating time: ");
  if(res == true){
```

```
    Serial.println("set successful!");
  }else{
    Serial.println("set failure!");
  }
}

void loop()
{
  static uint8_t       calibrated = 0;

  #ifdef CALIBRATE_PRESSURE
  if(calibrated == 0) {
    if(bme.iaqUpdate() == 0) {
      /*You can use an accurate altitude to calibrate sea level air pressure.
       *And then use this calibrated sea level pressure as a reference to obtain the calib
       *In this case,525.0m is chendu accurate altitude.
       */
      seaLevel = bme.readSeaLevel(525.0);
      Serial.print("seaLevel :");
      Serial.println(seaLevel);
      calibrated = 1;
    }
  }
  #else
    calibrated = 1;
  #endif

  if(calibrated) {
    uint8_t rslt = bme.iaqUpdate();
    if(rslt == 0) {
      Serial.println();
      Serial.print("timestamp(ms) :");
      Serial.println(millis());
      Serial.print("temperature(C) :");
      Serial.println(bme.readTemperature(), 2);
      Serial.print("pressure(Pa) :");
      Serial.println(bme.readPressure());
      Serial.print("humidity(%rh) :");
      Serial.println(bme.readHumidity(), 2);
      Serial.print("altitude(m) :");
      Serial.println(bme.readAltitude());
#ifdef CALIBRATE_PRESSURE
      Serial.print("calibrated altitude(m) :");
      Serial.println(bme.readCalibratedAltitude(seaLevel));
#endif
      Serial.print("gas resistance :");
      Serial.println(bme.readGasResistance());
      if(bme.isIAQReady()) {
        Serial.print("IAQ :");
        float iaq = bme.readIAQ();
        Serial.print(iaq);
        if(iaq < 50) Serial.println(" good");
        else if(iaq < 100) Serial.println(" average");
```

```
        else if(iaq < 150) Serial.println(" little bad");
        else if(iaq < 200) Serial.println(" bad");
        else if(iaq < 300) Serial.println(" worse");
        else Serial.println(" very bad");
      } else {
        Serial.println("IAQ not ready, please wait about 5 minutes");
        Serial.print("IAQ not ready, You will have to wait ");
        Serial.print((int)(305000-millis())/1000);
        Serial.println(" seconds");
      }
    }
  }
}
```

Result

without IAQ data



with IAQ data

```
IAQ :27.00 good

timestamp(ms) :362987
temperature(C) :28.00
pressure(Pa) :102952.00
humidity(%rh) :55.00
altitude(m) :-119.56
calibrated altitude(m) :495.91
```

Arduino library API list

- Create a bme object and write to the I2C address
  `DFRobot_BME68x_I2C bme(0x77);`
- Initializing the BME68x and Libraries
  `begin();`
- Start data conversion `startConvert();`
- Read converted data
  `update();`
- Supports IAQ reading
  `supportIAQ();`
- Read-converted data with IAQ
  `iaqUpdate();`
- Queries whether the IAQ conversion is complete, returns 1 if complete, 0 if not.
  `isIAQReady();`
- Get the temperature in °C, data with two decimal points
  `readTemperature();`
- Get the pressure in pa, data with two decimal points
  `readPressure();`
- Get the humidity in %rh with two decimal places.
  `readHumidity();`
- Get the resistance value of the gas resistor in ohm, data with two decimal points
  `readGasResistance();`
- Get the altitude in m with two decimal places.
  `readAltitude();`
- Acquisition of sea level barometric pressure reference, incoming elevation
  `readSeaLevel(float altitude);`
- Acquisition of corrected elevations to be passed into the sea level reference
  `readCalibratedAltitude(float seaLevel);`
- Set the target temperature of the heating layer (°C) and the heating duration (ms).

Temperature range 200~400°C

```
setGasHeater(uint16_t heaterTemp, uint16_t heaterTime);
```

Compatible test

MCU	Working well	Working wrong	Untested	special specification
FireBeetle-Board328P	√			IAQ is not supported
FireBeetle-ESP32	√			IAQ is not supported
FireBeetle-ESP8266	√			IAQ is supported
Leonardo/UNO/MEGA	√			IAQ is not supported

FAQ

More Document

- Schematic (https://dfimg.dfrobot.com/62b2fb5caa613609f271523c/wiki/2c01e9eacec738a878fb1631c8cfb115.pdf)
- Layout (https://dfimg.dfrobot.com/62b2fb5caa613609f271523c/wiki/2070b0ae2d901013cfc2358919bf45b6.pdf)
- BME688 Datasheet (https://dfimg.dfrobot.com/62b2fb5caa613609f271523c/wiki/602995d548af7b4461eb4ccbd60bb2c2.pdf)

Get one from DFRobot store (https://www.dfrobot.com/product-2871.html)